

Routing

A Single Page Application never actually reloads the browser — Angular's **Router** lets one page swap components based on the URL, giving you real navigation, bookmarkable links, and a working back button.

1 · THE PROBLEM

Our app needs more than one page

Users expect a real, multi-page experience — Dashboard, Products, Login, Profile — but a Single Page Application never actually reloads the browser. Angular's Router lets one page swap components based on the URL, giving you real navigation, bookmarkable links, and a back button that works.

2 · ROUTES

Mapping URLs to components

```
export const routes: Routes = [  
  { path: 'dashboard', component: DashboardComponent },  
  { path: 'products',   component: ProductsComponent },  
  { path: 'login',      component: LoginComponent },  
];
```

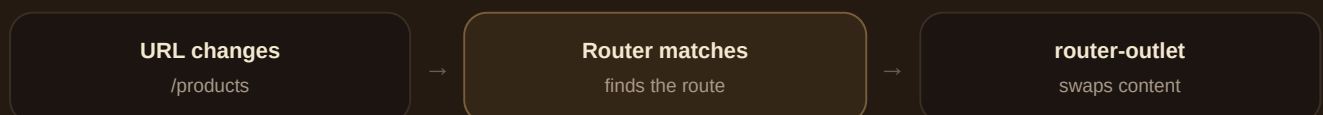
PATH	COMPONENT
/dashboard	DashboardComponent
/products	ProductsComponent
/login	LoginComponent

A route is simply a rule: "when the URL looks like this, show that component." Angular checks the list top to bottom and renders the **first** matching component.

3 · ROUTER & ROUTER-OUTLET

Where the matching component appears

```
<nav>...</nav>  
<router-outlet></router-outlet>
```



Think of `<router-outlet>` as a window frame that never moves — the Router just changes which picture is showing through it. The rest of the page — the navbar, footer, sidebar — stays exactly where it is while the outlet content swaps.

4 · TWO WAYS TO NAVIGATE

routerLink vs. the Router service

Declarative — routerLink

```
<a routerLink="/products">
  Products
</a>
```

Best for menus, nav bars, plain links the user clicks.

Programmatic — Router service

```
constructor(private router: Router) {}

goToLogin() {
  this.router.navigate(['/login']);
}
```

Best after logic runs first — e.g. redirect after a form submits.

Real-world usage: nav bars & tabs (routerLink) · "redirect after login" or "back to dashboard" (Router.navigate).

5 · ROUTE PARAMETERS

Passing data through the URL with :id

Problem: how do we show details for one specific product, like `/products/12`?

```
{ path: 'products/:id', component: ProductDetailComponent }

constructor(private route: ActivatedRoute) {}
ngOnInit() {
  const id = this.route.snapshot.paramMap.get('id');
}
```

The colon marks a dynamic segment — it can match `/products/12`, `/products/45`, any id at all — one route definition, endless product pages. `ActivatedRoute` is a service Angular injects into your component so it can read exactly which URL activated it, including any parameters like `:id`.

Real-world usage: product detail pages, user profiles, order details, any "show one item" screen.

6 · A REAL APP'S STRUCTURE

A tiny routes array powers an entire app

PATH	COMPONENT
<code>/dashboard</code>	<code>DashboardComponent</code>
<code>/products</code>	<code>ProductsComponent</code>
<code>/products/:id</code>	<code>ProductDetailComponent</code> (nested)
<code>/login</code>	<code>LoginComponent</code>
<code>/profile</code>	<code>ProfileComponent</code>

A dashboard, a products list, its nested detail page, a login screen, and a profile page — all from one small routes array.

Wiring up a mini app

```
export const routes: Routes = [
  { path: 'home', component: HomeComponent },
  { path: 'about', component: AboutComponent },
  { path: 'products/:id', component: ProductDetailComponent },
];

<a routerLink="/home">Home</a>
<a routerLink="/about">About</a>

ngOnInit() {
  const id = this.route.snapshot.paramMap.get('id');
}
// Visiting /products/7 shows product #7
```

`routerLink` is enough for Home and About because they're simple, static navigation targets — no logic needed first. The dynamic `:id` segment is what makes one route definition serve endless product pages.